

Lab4 理论测试

* 您的姓名：

 刘哲晗

* 学号：

24371277

* 1. 在 MOS 中，参数最多的系统调用需要接收 6 个参数。用户态程序执行 msyscall 指令触发系统调用时，对于这 6 个参数传递的规范，下列说法正确的是？（）

- ☐ A. 所有参数均直接压入用户栈中传递，内核从用户栈读取。
- ☐ B. 6 个参数全部存放在 \$a0-\$a5 寄存器中传递。
- ☐ C. 前 4 个参数存放在 \$a0-\$a3 寄存器中，后 2 个参数存放在用户栈中，且调用方无需为前 4 个参数在栈上预留空间。
- ☐ D. 前 4 个参数存放在 \$a0-\$a3 寄存器中，后 2 个参数存放在用户栈中，且调用方的栈帧必须至少预留 16 字节的空间。

* 2. 当系统陷入内核处理系统调用时，内核需要获取用户态传递的参数并将返回值交还给用户程序。关于这一过程，下列描述错误的是？（）

- ☐ A. 陷入内核后，SAVE_ALL 宏会将原用户进程的现场以 Trapframe 结构格式保存到内核栈的中。
- ☐ B. 内核中的 do_syscall 函数通过传入的 Trapframe 指针读取保存的用户进程现场，从而获取系统调用号和相关参数。

- ☐ C. 系统调用执行完毕后，内核直接将返回值写入用户进程的进程控制块（PCB）中，用户程序再从 PCB 中读取。
- ☐ D. 内核会将系统调用的返回值覆写到 Trapframe 结构保存的 \$v0 寄存器位置，随后恢复现场返回用户态。

* 3. 在实现进程间通信（IPC）机制时，调用 sys_ipc_recv 函数的进程会经历一系列状态变化。下列不属于 sys_ipc_recv 函数执行逻辑的是？（）

- ☐ A. 将当前进程控制块的 env_ipc_recving 标志设置为 1。
- ☐ B. 记录接收页面期望映射的目标虚拟地址到 env_ipc_dstva。
- ☐ C. 将当前进程状态设为 ENV_NOT_RUNNABLE 并放弃 CPU，内核重新选取下一个进程调度。
- ☐ D. 不断轮询检查发送方是否已经将数据写入自身的进程控制块中。

* 4. 进程 A 试图通过 sys_ipc_try_send 向进程 B 发送数据。如果此时进程 B 的 env_ipc_recving 标志为 0，系统会如何处理？（）

- ☐ A. 阻塞进程 A，直到进程 B 准备好接收为止。
- ☐ B. 将数据暂时存放在内核的缓冲队列中，等待进程 B 随后读取。
- ☐ C. 函数直接返回 -E_IPC_NOT_RECV 错误码，表明目标进程未处于接收状态。
- ☐ D. 强制唤醒进程 B，并将数据写入进程 B 的地址空间。

* 5. 在 fork 的实现中，关于内核态系统调用 sys_exofork 的行为，下列说法错误的是？（）

- ☐ A. 会为子进程分配一个新的进程控制块（PCB）。
- ☐ B. 会将父进程完整的用户地址空间（包括代码段和数据段）物理页复制一份给子进程。
- ☐ C. 会将父进程的运行现场（Trapframe）复制到子进程的 PCB 中。
- ☐ D. 会将子进程的现场中的 \$v0 寄存器修改为 0，以确保子进程从 fork 返回时值为 0。

* 6. 关于 fork 函数的调用与返回机制，下列说法正确的是？（）

- ☐ A. fork 在父进程中被调用两次，产生两个返回值。
- ☐ B. fork 在两个进程中分别被调用一次，产生两个不同的返回值。
- ☐ C. fork 只在父进程中被调用了一次，在两个进程中各产生一个返回值。
- ☐ D. fork 只在子进程中被调用了一次，在两个进程中各产生一个返回值。

* 7. 在用户态 fork 实现的 duppage 函数中，需要根据父进程中不同页面的权限进行不同的映射处理。对于带有 PTE_LIBRARY（共享）标志的页面，正确的处理方式是？（）

- ☐ A. 在父子进程中都加上 PTE_COW 标志，并取消 PTE_D 标志。
- ☐ B. 仅按只读权限原样映射给子进程，不做其他修改。
- ☐ C. 在父子进程中直接映射为相同物理页并保持可写状态，不添加 PTE_COW。
- ☐ D. 为子进程分配新物理页，拷贝数据后保持可写状态。

* 8. 在 fork 的用户态代码中，对 syscall_exofork 的包装函数被设计为内联（inline）函数。这样设计的根本原因是？（）

- ☐ A. 提高进程创建的速度，减少函数调用的开销。
- ☐ B. 避免在为栈页面设置写时复制（COW）保护之前，非内联函数返回修改栈帧导致另一进程的返回地址被意外覆盖。
- ☐ C. 使得 syscall_exofork 能够直接访问 fork 函数中的局部变量。
- ☐ D. MIPS 架构不允许在非叶函数中使用 syscall 汇编指令。

* 9. 当 CPU 捕获到 TLB Mod（页写入异常）时，内核的 do_tlb_mod 函数会进行异常转发。以下关于 do_tlb_mod 行为的描述，正确的是？（）

- ☐ A. 直接在内核态为触发异常的虚拟地址分配新的物理页并完成数据拷贝。
- ☐ B. 直接触发 panic，因为用户程序不应向没有 PTE_D 权限的页面写入数据。
- ☐ C. 将当前用户进程的执行现场（Trapframe）复制到当前用户（正常）栈上。

- ☐ D. 将 Trapframe 复制到专门的用户异常处理栈 (UXSTACKTOP) 上, 并将 EPC 设置为用户预先注册的异常处理入口 (假设已经设置异常处理入口)。

* 10. 在处理页写入异常时, 为何操作系统需要提供一个单独的“用户异常处理栈”, 而不是直接使用进程原有的正常用户栈? ()

- ☐ A. 正常用户栈的容量太小, 不足以完整保存 Trapframe 结构。
- ☐ B. 因为发生页写入异常的地址本身可能就位于正常用户栈上, 直接向其压栈可能导致异常无限触发, 造成死循环。
- ☐ C. 单独的异常栈位于内核空间, 能够保证异常处理数据的安全性。
- ☐ D. 硬件 MIPS 协处理器 CP0 规定异常处理必须切换物理栈。

* 11. 在 cow_entry 中, 当确认为写时复制页后, 系统需要分配一页新的物理内存。为何代码中必须将新分配的物理页临时映射到一个特殊的虚拟地址 (UCOW), 而不是直接将其映射到发生错误的虚拟地址 va? ()

- ☐ A. 直接映射到 va 会触发嵌套的 TLB Mod 异常, 导致异常处理死循环。
- ☐ B. 直接映射到 va 会立即覆盖原有的页表映射, 导致后续无法读取并拷贝原物理页中的数据。
- ☐ C. UCOW 是一段位于内核空间 kseg0 的特殊地址, 只有通过它才能保证数据拷贝时的原子性。
- ☐ D. 直接映射到 va 的操作受 MIPS 硬件限制, 必须通过修改临时寄存器绕过限制。

* 12. 在 MOS 中, 系统调用号 (如 SYS_mem_alloc) 的作用和传递方式, 下列描述正确的有 () 【多选题】

- ☐ A. 系统调用号是内核 do_syscall 中区分具体要执行哪个 sys_* 函数的依据。
- ☐ B. 所有的系统调用执行时, 在 MIPS 硬件层面均统一触发 8 号异常 (Syscall 异常)。
- ☐ C. 用户态包装函数调用 msyscall 时, 系统调用号作为第一个参数, 存放在 \$a0 寄存器中传入内核。
- ☐ D. 系统调用号在内核态处理完成后, 会被内核清零以防止信息泄露。

* 13. 关于微内核架构下 MOS 的写时复制 (COW) 和 fork 机制, 以下说法正确的有 () 【多选题】

- ☐ A. fork 时对普通可写页面的处理是:取消 PTE_D 标志, 并加上 PTE_COW 标志。
- ☐ B. 若要写时复制保护机制能正常工作, 必须先通过 sys_set_tlb_mod_entry 注册页写入异常处理函数。
- ☐ C. 为了遍历父进程的用户空间页表, 代码中使用了 vpt 和 vpd 指针, 这利用了页目录的自映射机制。
- ☐ D. 写时复制发生时 (触发 TLB Mod 异常), 物理内存的分配和数据拷贝操作是在内核态中由 do_tlb_mod 直接完成的。

* 14. 在实现 sys_mem_alloc 和 sys_mem_map 系统调用时, 都需要对传入的虚拟地址 (va) 进行合法性检查。以下关于这些检查的描述, 正确的有 () 【多选题】

- ☐ A. 传入的虚拟地址 va 必须小于 UTOP, 以防止用户程序越权操作内核空间或意外访问不应访问的内存区域。
- ☐ B. va 必须页对齐 (即 $va \% \text{PAGE_SIZE} == 0$) 。
- ☐ C. sys_mem_alloc 操作的目标进程、sys_mem_map 的源进程只能是当前调用进程。
- ☐ D. 只要 va 落在用户空间合法范围内, 即使调用进程的对应虚拟地址没有映射物理页, sys_mem_map 也能直接成功执行共享映射。

* 15. 进程 A 想要读取进程 B 地址空间中某一页的数据。在 MOS 操作系统的基础系统调用中, 最合适的实现方式是? ()

- ☐ A. 进程 A 调用 sys_mem_alloc 在 B 的空间分配内存。
- ☐ B. 进程 A 通过 IPC 与进程 B 进行通信。
- ☐ C. 进程 A 触发 TLB Mod 异常, 由内核自动拷贝 B 的页面到 A。
- ☐ D. 进程 A 利用 vpt 直接读取内核空间 kseg0 从而访问 B 的物理页。

提交

